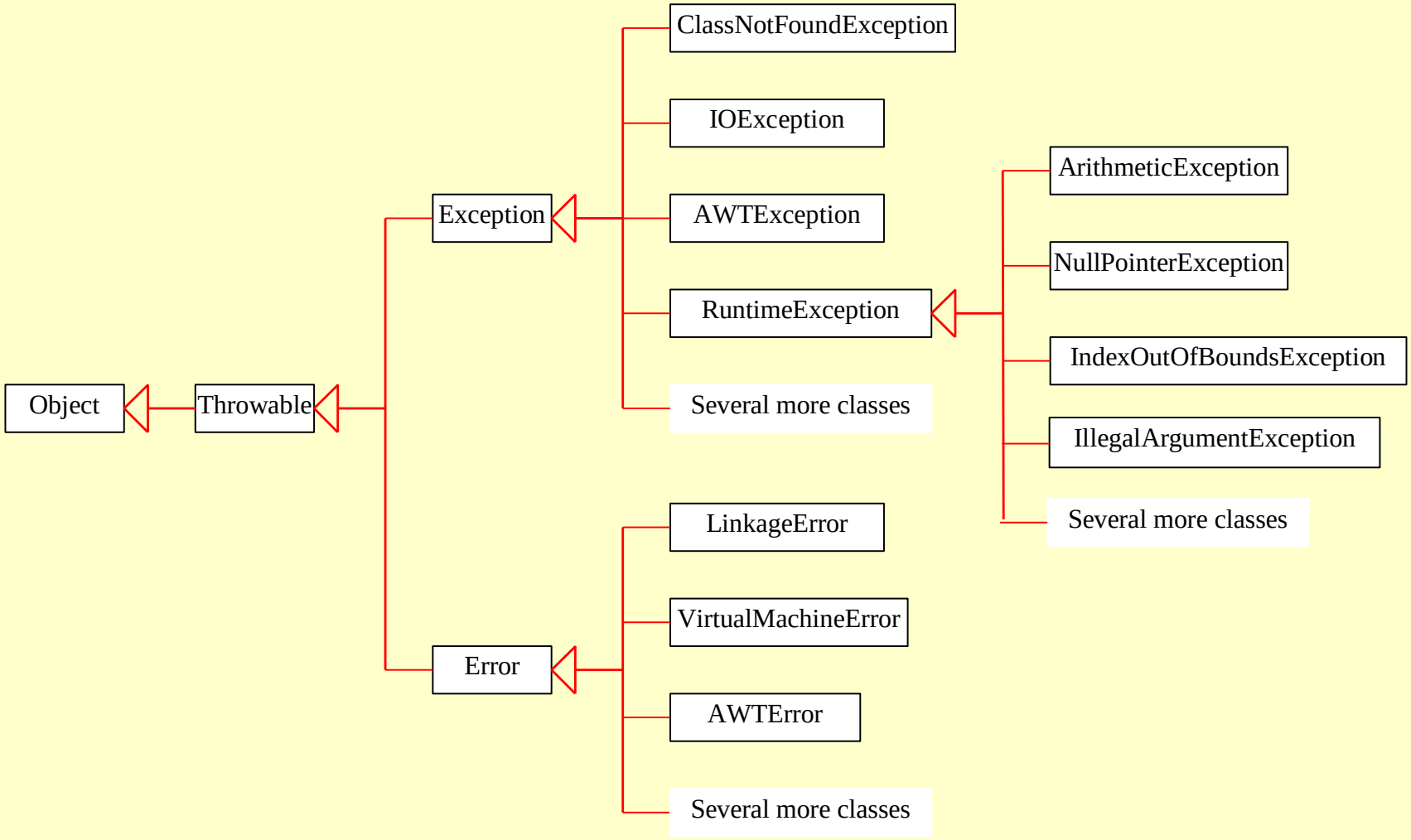
Exception Handling Part 2

Lecture 29

Based on Slides of Dr. Norazah Yusof

Exception Classes



ArithmeticException

- **Occurs when dividing an integer by zero.**
- **Question:**

  Write a program that reads a denominator value and then divide 100 with the denominator value.

  If the user enters 0 as the denominator value, then the exception is thrown. Write a catch block that handle this exception, by displaying a message: “Division by zero!”

Answer

```
1   import java.util.*;
2
3   public class TestDivide1{
4     public static void main(String args[]){
5        try {
6           Scanner scanner = new Scanner(System.in);
7
8           System.out.print ("Enter the denom value:");
9           int denom = scanner.nextInt();
10          System.out.println ("The value of 100/" + denom
11       +": "+ 100/denom);
12       }
13          catch (ArithmeticException ex) {
14             System.out.println ("Division by zero error!");
15        }
16     }
17  }
```

InputMismatchException

- **Occurs when user enters a string for an integer variable.**
- **Question:**

  Modify the program that handle the exception when user enters a string i.e “r” as a denominator value.

  Write the appropriate catch block that handle this exception, by displaying a message: “Wrong input data!”

Answer

```
1   import java.util.*;
2   public class TestDivide2{
3     public static void main(String args[]){
4        try {
5           Scanner scanner = new Scanner(System.in);
6
7           System.out.print ("Enter the denom value:");
8           int denom = scanner.nextInt();
9           System.out.println ("The value of 100/" + denom
10     +": "+ 100/denom);
11     }
12     catch (ArithmeticException ex) {
13           System.out.println ("Division by zero error!");
14      }
15
16      catch (InputMismatchException ex) {
17         System.out.println ("Wrong input data!");
18      }
19    }
20  }
```

printStackTrace(),getStackTrace(), getMessage()

- Class Throwable offers methods such as:

1. printStackTrace() method that output the standard error stream the stack trace.
2. getStackTrace() method that retrieves stack trace information that might be printed by printStackTrace.
3. getMessage() method that returns the descriptive string stored in an exception.

Self Test

- **Question:**

  Modify the previous program that invoke the printStackTrace() method when any of the exception occurs.

Example

```
1   import java.util.*;
2   public class TestDivide2{
3     public static void main(String args[]){
4        try {
5           Scanner scanner = new Scanner(System.in);
6
7           System.out.print ("Enter the denom value:");
8           int denom = scanner.nextInt();
9           System.out.println ("The value of 100/" + denom
10     +": "+ 100/denom);
11     }
12     catch (ArithmeticException ex) {
13           ex.printStackTrace();
14      }
15
16      catch (InputMismatchException ex) {
17         ex.printStackTrace();
18      }
19    }
20  }
```

Throwing Exceptions

- **When the program detects an error, the program can create an instance of an appropriate exception type and throw it.**
- **This is known as *throwing an exception*. Here is an example,**

```
throw new TheException();

TheException ex = new TheException();
throw ex;
```

Note: keyword `throws (throw)` to declare (throw) an exception

Write own exception

- **Question:**

  Modify the program that throw a new exception object named NumberFormatException with “Not positive denom” as its parameter value. This exception occurs when user enters a negative value for denominator. Write the appropriate catch block that handle this exception, by displaying its message.

Answer

```
1   import java.util.*;
2   public class TestDivide4{
3     public static void main(String args[]){
4        try {
5           Scanner scanner = new Scanner(System.in);
6
7             System.out.print ("Enter the denom value:");
8           int denom = scanner.nextInt();
9           if (denom<0)
10            throw new NumberFormatException("Not a positive denom!!");
11          System.out.println ("The value of 100/" + denom +": "+
12    100/denom);
13    }
14    catch (ArithmeticException ex) {
15           System.out.println ("Division by zero error!");
16     }
17
18     catch (InputMismatchException ex) {
19        System.out.println ("Wrong input data!");
20     }
21
22     catch (NumberFormatException ex) {
23         System.out.println (ex.getMessage());
24     }
25   }
26  }
27  }
```

Propagate the Exception Handler in different methods

- **If one of the statement inside the try block throws an exception, JVM starts the process of finding the code to handle the exception.**
- **The code that handles the exception is called the *exception handler*,**
  - **It is found by propagating the exception backward through a chain of method calls, starting from the current method.**

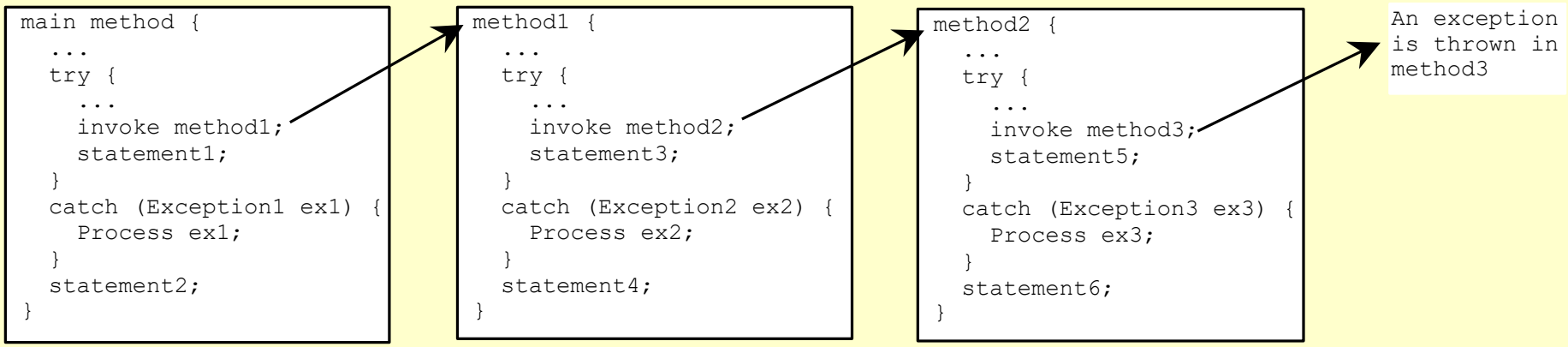
Propagate the Exception Handler in different methods

- **Each catch block is examine in turn, from first to last, to determine whether the type of exception is an instance of exception class.**
  - **If so, the exception object is assigned to the variable declared and the code in catch block is executed.**
  - **If not, Java exit this method and passes the execution to the method that invoked the method, and continue the same process to find a handler.**

Uncaught Exceptions in a method

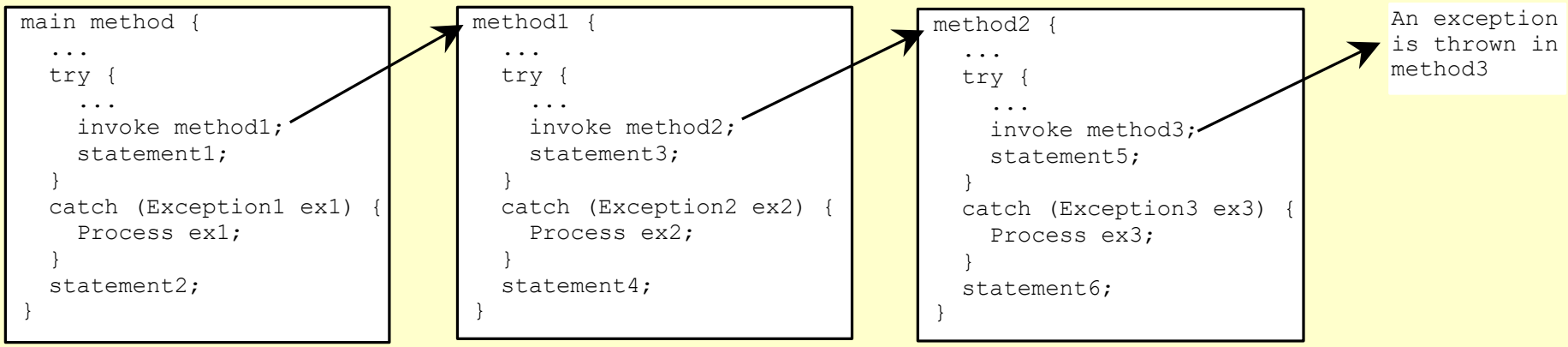
- **If there is no exception handler inside the method:**
  - **control of the program is passed to the previous method in the call stack.**
  - **If that method has no exception handler, then control is passed again, up the call stack, to the previous method.**
- **If control reaches the `main` method:**
  - **the main method must either handle the exception, or**
  - **the program is halted and the default exception handler handles the exception.**

Propagate the Exception Handler in different methods



Suppose the main method invoke method1, method1 invokes method2, method2 invoke method3, and an exception occurs in method3.

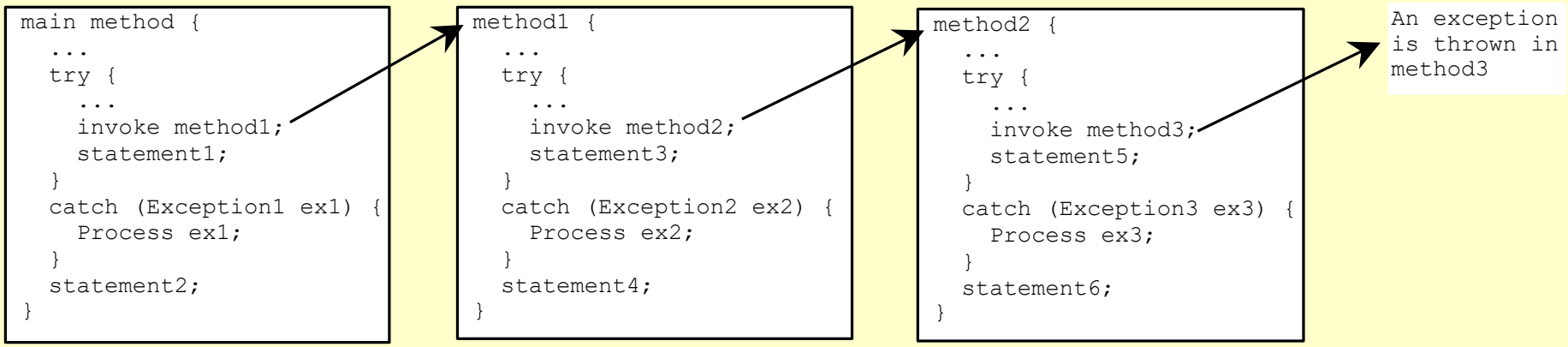
Propagate the Exception Handler in different methods



Suppose the main method invoke method1, method1 invokes method2, method2 invoke method3, and an exception occurs in method3.

If methof3 cannot handle the exception, method3 is aborted and the control is returned to method2.
If the exception type is Exception3, it is caught by the catch block for handling exception ex3 in method2. statement5 is skipped, and statement6 is executed.

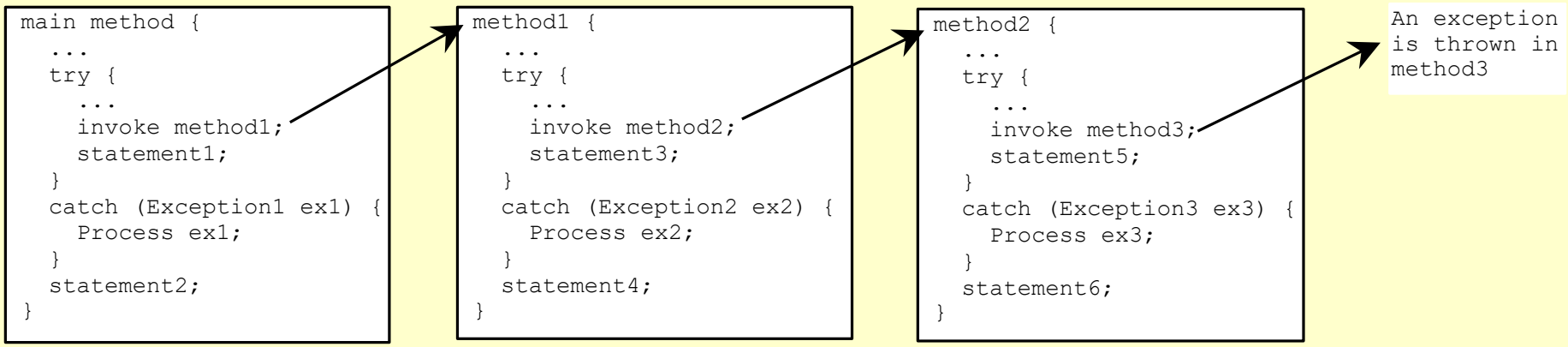
Propagate the Exception Handler in different methods



Suppose the main method invoke method1, method1 invokes method2, method2 invoke method3, and an exception occurs in method3.

If the exception type is Exception2, method2 is aborted, the control is returned to method1, and the exception is caught by the catch block for handling exception ex2 in method1. statement3 is skipped, and statement4 is executed.

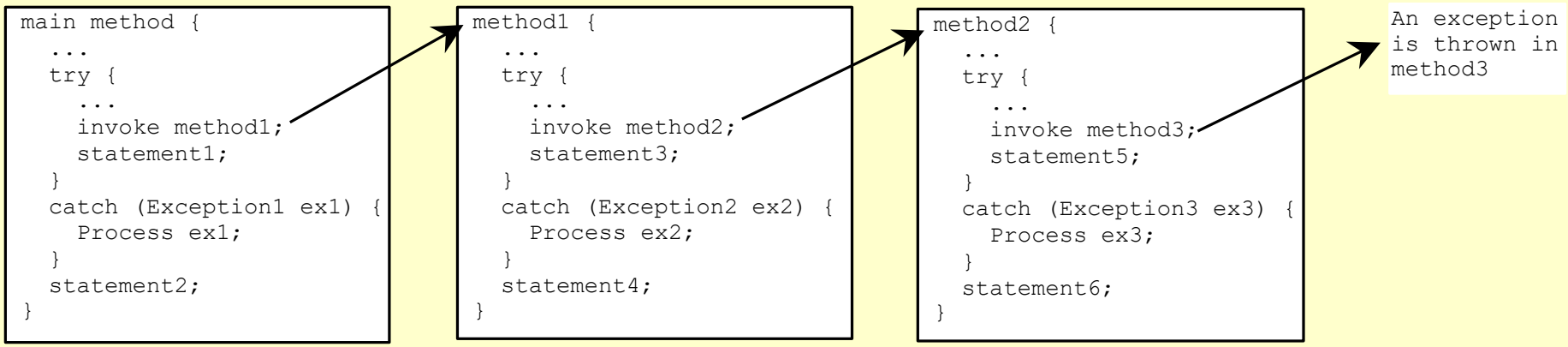
Propagate the Exception Handler in different methods



Suppose the main method invoke method1, method1 invokes method2, method2 invoke method3, and an exception occurs in method3.

If the exception type is Exception1, method1 is aborted, the control is returned to main method, and the exception is caught by the catch block for handling exception ex1 in main method. statement1 is skipped, and statement2 is executed.

Propagate the Exception Handler in different methods



Suppose the main method invoke method1, method1 invokes method2, method2 invoke method3, and an exception occurs in method3.

If the exception type is not Exception1, Exception2, or Exception3, the exception is not caught and the program terminates. Statement1 and statement2 are not executed.

Propagate the Exception Handler in different method

```
public class Pd
{  public void metod1(int i, int j)
   {    try
        {metod2(i,j);  }
        catch(ArithmeticException e)
        {  System.out.println("i=" +i+ "\tj=" +j);
           e.printStackTrace();
           System.out.println("This is your problem:" +e.getMessage());
        }}
   public void metod2(int i, int j) throws ArithmeticException
   {    System.out.println("metod2 bermula");
        j = i/j;
        System.out.println("metod2 berakhir");}

   public static void main (String[] a)
   {  Pd p = new Pd();
        p.metod1(1,1); // tiada exception yang berlaku
        p.metod1(1,0); // exception berlaku di metod2        }}
```

Order of the Exception Handling

- **The order of which exceptions are specified in catch clauses is important.**
- **Compilation error will result if a catch clause for a superclass type appears before a catch clause for a subclass type.**
- **For example:**

Wrong Order of the Exception Handling

```
try{
. . .
}
catch (Exception ex){
  …
}
catch (RunTimeException ex){
  …
}
```

RunTimeException class is a subclass of Exception. Error occurs because superclass appears before a catch clause for the subclass type.

Correct Order of the Exception Handling

```
try{
. . .
}
catch (RunTimeException ex){
 …
}
catch (Exception ex){
 …
}
```

RunTimeException class is a subclass of Exception.

Cautions When Using Exceptions

- **Exception handling separates error-handling code from normal programming tasks, thus making programs easier to read and to modify.**
- **Be aware, however, that exception handling usually requires more time and resources because it requires**
  - **instantiating a new exception object,**
  - **rolling back the call stack, and**
  - **propagating the errors to the calling methods.**

When to Use Exceptions

When should you use the try-catch block in the code? You should use it to deal with unexpected error conditions. Do not use it to deal with simple, expected situations. For example, the following code

```
try {
  System.out.println(refVar.toString());
}
catch (NullPointerException ex) {
  System.out.println("refVar is null");
}
```

When to Use Exceptions

is better to be replaced by

```
if (refVar != null)
  System.out.println(refVar.toString());
else
  System.out.println("refVar is null");
```

Creating Custom Exception Classes

- Use the exception classes in the API whenever possible.
- Create custom exception classes if the predefined classes are not sufficient.
- Declare custom exception classes by extending Exception or a subclass of Exception.

Creating your own Exception class

Steps:

a) **Define the Exception class as a subclass to the Exception class:**

```
public class MyException extends Exception
```

b) **Build a constructor**

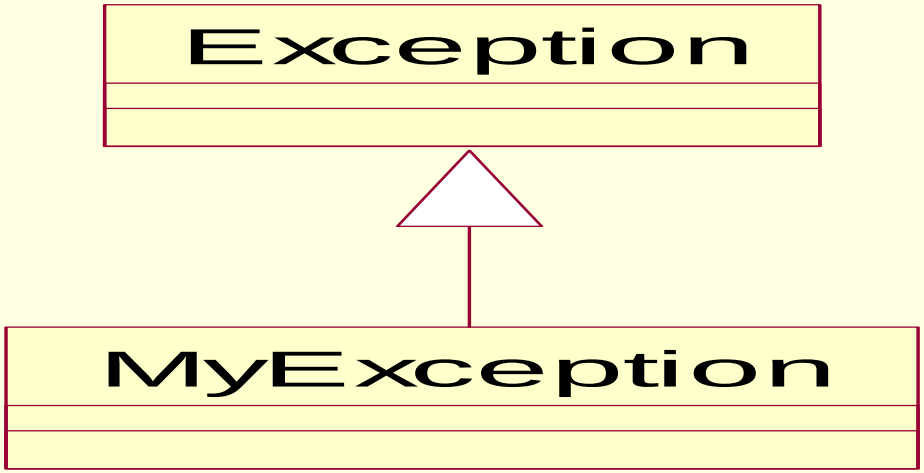
```
public MyException(int i)
{
value = i;
}
```

c) Override a super class method

```
public String getMessage()
{
    return message;
}
```

d) Steps a)+ b) + c)

Overall Picture



```
public class MyException extends Exception {

  private int value;
  private String message;

  public MyException(int i) {
    value = i;
    message = new String("MyException occurs at
value: " + value);
  }
 public String getMessage() {
    return message;
  }
}
```

How to use the user-defined exceptions?

```
1  import java.util.*;
2  public class TestMyException {
3    public static void main(String[] args) {
4        Scanner inp = new Scanner(System.in);
5        try {
6           System.out.print ("Enter a value:");
7          int i = inp.nextInt();
8          if(i == -999)
9            throw new MyException(i);
10         else if (i == 123)
11           throw new MyException(i);
12         else
13           System.out.println ("The valid value is "+i);
14          }
15       catch (MyException ex) {
16         System.out.println(ex.getMessage());
17          }
18    }
19 }
```